

Objectives :

- Learn About How Computers Work
- Introduction to Programming Languages
- Introduction to Node.js
- User Input and Output

How Computers Work

We all have computers, and we use them for a variety of purposes, including watching videos, playing games, performing mathematical calculations, communicating with friends, and many other applications. But the fundamental question is: how do these devices actually work? The answer lies in their electrical nature. Computers are essentially electrical devices that perform all calculations using electrical signals. The central processing unit (CPU), often referred to as the "brain" of the computer, executes these calculations. To temporarily store data while the computer is in operation, it relies on memory, specifically Random Access Memory (RAM).

Crucially, all information within a computer is represented by electrical signals. This includes data stored in memory and the instructions that the CPU executes. These electrical signals exist in one of two distinct states: presence (typically represented by the digit '1') or absence (represented by '0').

With this binary representation in mind, we can understand that CPU instructions are essentially sequences of 1s and 0s. This sequence of binary digits is known as machine code, which is the most fundamental level of programming language that the CPU can directly understand.

Introduction to Programming Languages

Programming languages are tools that were developed to facilitate communication with computers. Instead of writing instructions directly in binary code, which can become incredibly cumbersome for large programs, we can use programming languages with their more user-friendly syntax. This simplifies the coding process, making it easier to read, understand, and debug code.

However, computers cannot directly understand the syntax of these high-level programming languages. To bridge this gap, we use a program called a compiler or an interpreter. These tools translate the code into machine code, a low-level language consisting of binary instructions that the computer can execute.

Introduction to Node.js

Introduction

Node.js is a modern, open-source runtime environment that allows us to run JavaScript code outside the web browser. Built on Chrome's V8 JavaScript engine, Node.js enables developers to build scalable network applications using JavaScript on the server side.

It is known for its event-driven, non-blocking I/O model, which makes it lightweight and efficient for building fast, real-time applications. Node.js is particularly popular for creating web servers, APIs, and microservices, but it can also be used for scripting tasks, automation, and much more.

Node.js files typically have a `.js` extension. These files contain JavaScript code that runs directly on the machine using Node.js.

Node.js Installation

Before starting with Node.js, we need to install it on our system. Node.js comes bundled with `npm` (Node Package Manager), which is useful for managing external libraries and dependencies.

We can download and install Node.js from the official website: <https://nodejs.org/>

After installation, we can verify it by running these commands in the terminal:

```
node --version
npm --version
```

These commands will display the installed versions of Node.js and npm, confirming the setup.

Running our First Program

Once Node.js is installed, we can run our first JavaScript program using Node.

Every Node.js application starts with a JavaScript file, typically containing a `console.log()` statement to output information to the console.

```
console.log('Hello, world!');
```

To run this program:

1. Save the code in a file called `hello.js`.
2. Open the terminal and navigate to the folder where `hello.js` is saved.
3. Run the command:

```
node hello.js
```

This will execute the program and display `Hello, world!` in the console.

Variables

Variables are fundamental building blocks in Node.js applications. They act like labeled containers that store data. Variables can hold numbers, text, boolean values, and even more complex types like objects and arrays.

In Node.js (and JavaScript in general), we declare variables using `let`, `const`, or `var`:

- `let` and `const` are modern keywords preferred over the older `var`.
- Use `let` when the variable value will change.
- Use `const` when the value should stay constant.

```
let city = 'Algiers';  
const country = 'Algeria';
```

Once declared with `const`, we cannot reassign a new value to the variable.

Data Types

We use variables to store data, including numeric values, text (strings), lists (arrays), objects, and dates. JavaScript provides different data types to efficiently store and work with various kinds of information. JavaScript data types are divided into two main categories Primitive Data Types and Non-Primitive Data Types

Primitive Data Types

In JavaScript, we work with primitive data types as our most basic building blocks. These types have three crucial behaviors we need to understand:

1. **Direct Memory Storage** - When we create primitive values, they're stored directly in the variable's memory location
2. **Immutability** - Once we create them, we can't modify primitive values themselves
3. **Value-based Operations** - When we try to "change" a primitive, we're actually creating a brand new value in memory

Number

The first primitive type we encounter in JavaScript is the `number` type. We use this type to represent all numeric values, including both integers and floating-point numbers.

```
let age = 23;
let pi = 3.14;
let negative_number = -5;
```

Bigint

`BigInt` is a primitive data type we use to represent integer values that are too large to be stored safely as regular `Numbers`. We create `BigInts` by appending `n` to the end of an integer or using the `BigInt()` constructor.

```
let number1 = 12345678901234567890n
let number2 = BigInt("9007199254740991")
```

String

The `string` type is used to store and manipulate textual data. We can create strings by enclosing our text in either single quotes (`"`), double quotes (`""`), or backticks (```) for template literals.

```
let greeting = "Hello World";
let name = 'Alice';
```

Template literals (enclosed in backticks ```) allow us to embed expressions and variables directly into strings. To inject values we create strings using ``` then we insert variables or expressions with `${}`

```
let name = "Alice";
let greeting = `Hello, ${name}!`;
```

The `greeting` variable will have as value `Hello, Alice`

Boolean

Booleans are used to store the results of logical operations, with only two possible values: `true` or `false`.

```
let is_adult = false;
```

JavaScript automatically converts any value to a boolean when used in a logical context. The result depends on whether the value is "truthy" or "falsy".

Truthy Values

```
true, "text", 1, [], {}, 3.14, Infinity, -1, "false"
```

Falsy Values

```
false, 0, "", null, undefined, NaN, 0n
```

Undefined

`undefined` represents a variable that has been declared but not yet assigned a value. It is JavaScript's default state for uninitialized variables.

```
let number;
```

Here type of number will be `undefined`

Null

`null` is a special primitive value that represents the intentional absence of any object value. We use it to explicitly indicate that a variable should have "no value" or "empty value."

```
let user = null;
```

Non-Primitive Data Types

Unlike primitive types, non-primitive types (objects) are mutable and store references to memory locations rather than direct values. They share these key behaviors:

1. **Reference Storage** - Variables store pointers to memory locations
2. **Mutability** - We can modify their contents without creating new references
3. **Pass-by-Reference** - Assignments/copies share the same underlying data

Object

We use objects to store collections of key-value pairs, where:

- **Keys** are always strings.
- **Values** can be any data type including other objects, functions, or primitives

To create an object, we place key-value pairs inside curly braces `{ }`. We separate each key from its value with a colon (`:`), and multiple pairs with commas (`,`):

```
const user = {  
  name: "Alex",  
  age: 30,  
  isAdmin: true,  
  address: {  
    city: "Berlin",  
    country: "Germany"  
  }  
}
```

There are two ways to access elements inside an object:

1. Dot Notation

We start with the object name, followed by a dot (`.`), then the key:

```
user.name;
```

2. Bracket Notation

We start with the object name, followed by square brackets `[]` containing the key (as a string):

```
user["name"];
```

Arrays

We use arrays to store ordered collections of data where:

- **Elements** can be any data type (including other arrays/objects)
- **Indexes** are zero-based integer positions
- **Length** dynamically adjusts as we add/remove items

To create an array, we place values inside **square brackets** `[]`. We separate the values with commas (`,`):

```
const mixedArray = [
  "text",
  42,
  true,
  { id: 1 },
  ["nested"]
];
```

We access values inside an array by using indexes. The first element has index 0. For example, to access the value 42 we use `mixedArray[1]` .

Functions

The final non-primitive data type is the **function**. We use functions to create reusable blocks of code that execute only when called.

```
function greet(name) {
  return `Hello, ${name}!`;
}
greet("Ali");
```

Constants

Constants are variables whose values cannot be changed once assigned. We create constants by using the keyword `const`

```
const pi = 3.14159;
const maxUsers = 100;
```

Type Conversion

JavaScript offers flexible ways to convert between types.

To Number

We can convert string variable to number using `Number()` if the function couldn't convert it will return `NaN`

```
let value = Number("42");
let safeValue = parseInt("42");
```

To String

We can Convert other data types to string using `.toString()` or `String()`

```
let message = (100).toString();
let status = String(true);`
```

To Boolean

We can also convert other types to boolean. Depending on their values, we will get either `true` or `false`.

```
let isEmpty = Boolean("hello"); // true
let isZero = Boolean(0); // false
```

Now for the **Node.js (JavaScript) truthy/falsy table:**
falsy table:

Value	Boolean Equivalent	Description
false	false	The boolean false
0	false	The number zero
-0	false	Negative zero
0n	false	BigInt zero
"" (empty string)	false	Empty string
null	false	Null value
undefined	false	Undefined value
NaN	false	Not-a-Number

Everything **else** in JavaScript (and Node.js) is **truthy**, including:

Value	Boolean Equivalent	Description
true	true	Boolean true
{}	true	An empty object
[]	true	An empty array
"0"	true	A non-empty string

Value	Boolean Equivalent	Description
"false"	true	A non-empty string
function() {}	true	Any function
new Date()	true	Date object
Infinity	true	Positive infinity
-Infinity	true	Negative infinity

Comments

Comments are lines of code that the computer will ignore and not execute. JavaScript has two types of comments. They help us add explanations and documentation to our scripts.

Single-line Comments

We create them using `//`; everything that comes after it will be a comment until the next line.

```
// This is a single-line comment
console.log("Hello, world!"); // Prints greeting
```

Multi-line Comments

If we want a multi-line comment, we use `/* */`. Everything inside will be considered a comment.

```
/* This is a multi-line comment.
Useful for longer explanations. */
```

User Input and Output

Interacting with the user is essential in most applications. Node.js provides built-in modules to handle input and output.

Output

When we want to display information or messages to the console we use `console.log()`.

```
console.log("Hello!");
```

We can also use `process.stdout.write()` to write without a newline:

```
process.stdout.write("Hi");  
process.stdout.write(" there!");
```

Escape Characters

Escape Characters are special characters that help use to write special ond hard characters on our string likes tabs and line break

Escape Character	Meaning	Example
<code>\n</code>	Newline	"Line 1\nLine 2"
<code>\t</code>	Tab space	"Name:\tAlice"
<code>\\</code>	Backslash	"C:\\Users\\Alice"
<code>\"</code>	Double quote	"He said, \"Hello!\""
<code>\'</code>	Single quote	'It\'s fine.'

```
console.log("She said, \"Welcome!\"\nLet's start learning Node.js.\n");
```

Output:

```
She said, "Welcome!"  
Let's start learning Node.js.
```

Input

Taking user input in Node.js is a little bit more complex than in some other programming languages.

First, we import the `readline` module by using `require('readline')` , this gives us access to special functions for handling user input, next, we create an interface using `readline.createInterface()` , when we create it, we connect it to `process.stdin` (to read what the user types) and `process.stdout` (to show messages to the user).

After setting up the interface, we use `rl.question()` to ask a question to the user.

The `question()` method needs two things:

- The text of the question we want to ask.
- A callback function that will run after the user answers.

Inside this callback function, we can access the user's input.

Finally, once we finish working with the user's input, we close the interface by calling

`rl.close()` .

```
const readline = require('readline');

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

rl.question('What is your name? ', (name) => {
  console.log(`Hello, ${name}!`);
  rl.close();
});
```

Tasks

Task 1

Write a program that reads the radius of a circle from the user and then displays its surface area.

Task 2

Develop a temperature converter that convert from celsius to fahrenheit